INTERVIEW PREP

Data Structures & Algorithms (Interview Prep)

A visual, pattern-first guide to the data structures, algorithms, and problem-solving frameworks that win coding interviews.

dsa · algorithms · data-structures · interview-prep · big-o · patterns

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. How to Approach a Coding Interview
2. Big-O & Complexity Analysis
3. Arrays & Strings
4. Hash Maps & Sets
5. Stacks & Queues
6. Linked Lists
7. Trees & BSTs
8. Heaps / Priority Queues
9. Graphs: BFS & DFS
10. Tries
11. Core Patterns
12. Sorting & Searching
13. Pattern → Signals → Example Problems
14. Interview Questions & Answers
15. Study Plan & Further Learning

A fast, interview-focused tour of the data structures, algorithms, and problem-solving patterns that show up again and again in technical interviews. Start here with the mindset and the math.

How to Approach a Coding Interview

Most candidates lose points not because they lack knowledge, but because they jump straight to code. Interviewers are grading your **process**: how you clarify, how you reason about trade-offs, and

how you verify. A repeatable framework keeps you calm and makes your thinking visible.

The UMPIRE framework

UMPIRE is a durable checklist you can run on any problem. It maps cleanly onto the shorter *clarify → plan → code → test* loop that experienced engineers use instinctively.

U — UNDERSTAND

Restate the problem in your own words. Ask about input size, types, duplicates, empty/negative cases, and expected output format.

M — MATCH

Which known category is this? Sorting, graph search, DP, two pointers? Pattern-matching narrows the solution space fast.

P — PLAN

Sketch the approach in plain language or pseudocode *before* typing. Agree on it with the interviewer.

I — IMPLEMENT

Write clean, idiomatic code. Narrate as you go so your reasoning is auditable.

R — REVIEW

Re-read your code line by line. Hunt for off-by-one errors, null handling, and mutation bugs.

E — EVALUATE

State time and space complexity. Walk through examples and edge cases out loud.

Clarify -> **Examples** -> **Plan** -> **Code** -> **Test** -> **Optimize**

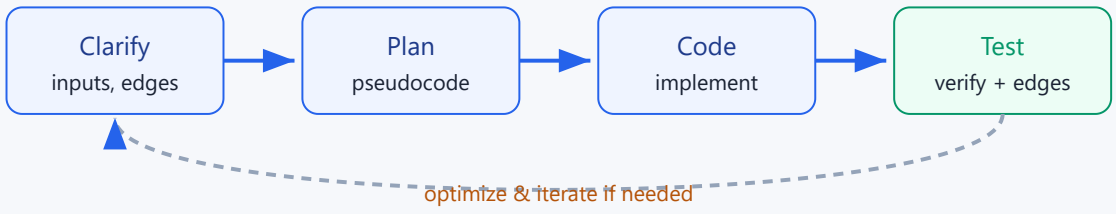


Fig 1. The core interview loop — clarify before you code, test before you declare done.

> ✦ **Tip**
>
> Spend the first 2–3 minutes only on understanding and examples. A concrete worked example (including an edge case) catches most misunderstandings before they cost you.

> ⚠ **Watch out**
>
> Don't optimize prematurely. Get a correct brute-force solution verbalized first, then improve. A working $O(n^2)$ beats a broken $O(n)$.

Communication checklist

- **Think out loud.** Silence reads as being stuck.
- **Name your complexity** as soon as you have an approach.
- **Ask before assuming** — can inputs be empty? sorted? contain duplicates or negatives?
- **Dry-run** your final code on one normal case and one edge case.

Big-O & Complexity Analysis

Big-O describes how an algorithm's running time or memory grows as the input size `n` grows. We drop constants and lower-order terms and keep the *dominant* growth term, because that is what matters at scale.

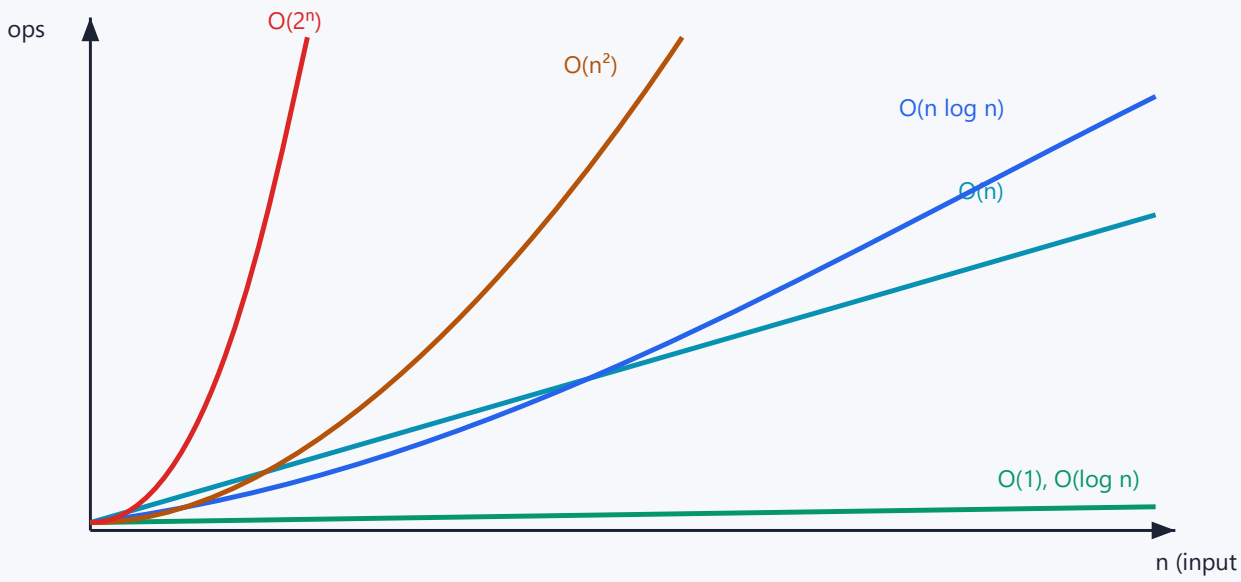


Fig 2. Growth curves. Flatter is better — small differences in order explode at large n.

Complexity cheat-sheet

Notation	Name	Example operation	Verdict
O(1)	Constant	Array index, hash lookup, stack push	Excellent
O(log n)	Logarithmic	Binary search, balanced-BST lookup	Excellent
O(n)	Linear	Single scan, linked-list traversal	Good
O(n log n)	Linearithmic	Merge sort, heap sort, efficient sorts	Good
O(n^2)	Quadratic	Nested loops, naive pair comparison	Caution
O(2^n)	Exponential	Naive recursion over subsets	Bad
O(n!)	Factorial	Brute-force permutations (travelling salesman)	Bad

> **Note**
>
> **Amortized** complexity averages cost over a sequence of operations. A dynamic array's `append` is O(1) amortized even though occasional resizes cost O(n), because resizes are rare and double capacity.

Analyzing your own code

- **Sequential** statements add: O(a) + O(b) = O(a + b).
- **Nested** loops multiply: a loop inside a loop over n is $O(n^2)$.
- **Halving** the input each step gives a log n factor.
- **Space** counts auxiliary memory: the recursion call stack, extra arrays, and hash maps all add up.

✓ **Rule of thumb**
For n up to ~10^5 an O(n log n) solution runs comfortably within a one-second limit. If n is tiny (≤ 20), an exponential solution may be acceptable — and sometimes expected.

Arrays & Strings

Contiguous, index-addressable sequences — the most common input type in interviews. Strings behave like immutable arrays of characters in most languages, including Python.

STRENGTHS

O(1) random access by index. Cache-friendly contiguous memory. Simple to reason about.

WEAKNESSES

Insertion/deletion in the middle is O(n) (shifting). Fixed-size in low-level languages; Python lists auto-grow (amortized O(1) append).

Operation	Time	Note
Index access `a[i]`	O(1)	Direct address
Append (end)	O(1) amortized	Occasional resize
Insert/delete middle	O(n)	Shift elements
Search (unsorted)	O(n)	Linear scan
Search (sorted)	O(log n)	Binary search

In-place reversal is a canonical warm-up. It uses O(1) extra space with two indices converging from the ends.

PYTHON

```python
def reverse_in_place(a):
    i, j = 0, len(a) - 1
    while i < j:
        a[i], a[j] = a[j], a[i]
        i += 1
        j -= 1
    return a

# Strings are immutable in Python, so build a new one:
def reverse_string(s):
    return s[::-1]
```

> **Tip**
> When a problem says "sorted array", think binary search or two pointers. When it says "subarray" or "substring", think sliding window or prefix sums.

Prefix sums

Precomputing cumulative sums turns repeated range-sum queries from O(n) each into O(1) each.

PYTHON

```python
def build_prefix(nums):
    prefix = [0] * (len(nums) + 1)
    for i, x in enumerate(nums):
        prefix[i + 1] = prefix[i] + x
    return prefix

def range_sum(prefix, lo, hi):   # inclusive [lo, hi]
    return prefix[hi + 1] - prefix[lo]
```

Hash Maps & Sets

A hash map (dict) stores key→value pairs with average O(1) lookup, insert, and delete. It hashes the key to a bucket index; collisions are resolved by chaining or open addressing. A set is the same structure storing only keys.

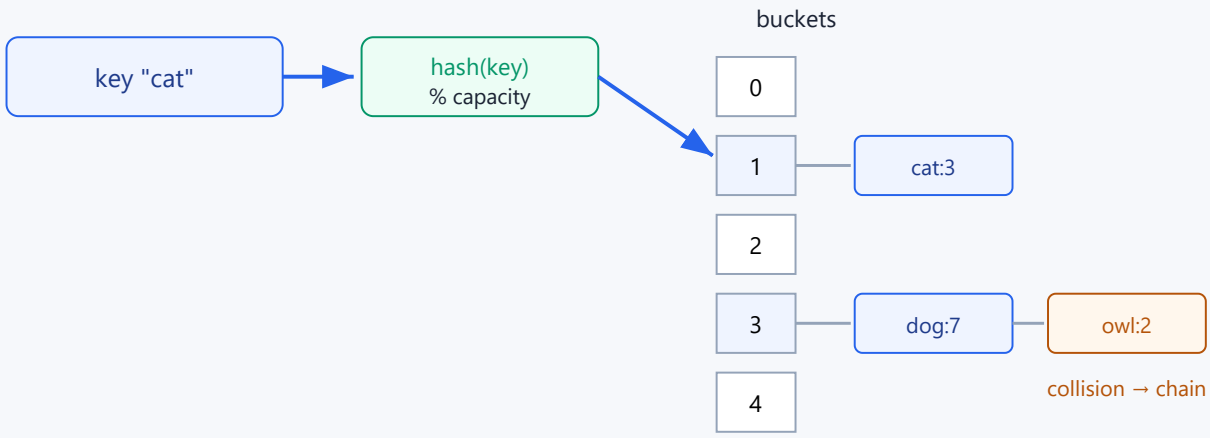


Fig 3. A hash table: keys hash to bucket indices; collisions chain within a bucket.

Operation	Average	Worst
Insert / lookup / delete	O(1)	O(n) (all keys collide)
Iterate all entries	O(n)	O(n)

PYTHON

```python
# Classic: two-sum in one pass with a hash map
def two_sum(nums, target):
    seen = {}                       # value → index
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []

# Set for O(1) membership / de-duplication
unique = set([1, 2, 2, 3])          # {1, 2, 3}
```

⚠ **Watch out**

Keys must be hashable (immutable). Lists and dicts can't be keys; use tuples or frozensets instead.

Stacks & Queues

A **stack** is LIFO (last-in, first-out) — think a stack of plates. A **queue** is FIFO (first-in, first-out) — think a line at a counter. Both support O(1) push and pop at their respective ends.

STACK USES

Undo history, matching brackets, DFS, expression evaluation, backtracking, call stacks.

QUEUE USES

BFS, task scheduling, buffering, producer/consumer pipelines, rate limiting.

PYTHON

```python
from collections import deque

stack = []
stack.append(1)       # push
stack.append(2)
top = stack.pop()     # 2  (LIFO)

queue = deque()
queue.append(1)       # enqueue
queue.append(2)
front = queue.popleft()  # 1  (FIFO)

# Valid parentheses with a stack
def is_valid(s):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for c in s:
        if c in pairs:
            if not stack or stack.pop() ≠ pairs[c]:
                return False
        else:
            stack.append(c)
    return not stack
```

Tip

Use `collections.deque` for queues, not a plain list — `list.pop(0)` is O(n), while `deque.popleft()` is O(1).

Linked Lists

A sequence of nodes where each node holds a value and a pointer to the next node. Unlike arrays, there is no random access, but insertion/deletion at a known node is O(1) because no shifting is required.

Operation	Array	Linked list
Access by index	O(1)	O(n)
Insert/delete at head	O(n)	O(1)
Insert/delete at known node	O(n)	O(1)
Search	O(n)	O(n)

PYTHON

```python
class ListNode:
    def __init__(self, val=0, nxt=None):
        self.val = val
        self.next = nxt

# Iterative reversal: the single most-asked linked-list question
def reverse_list(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next      # save next
        cur.next = prev     # flip pointer
        prev = cur          # advance prev
        cur = nxt           # advance cur
    return prev             # new head
```

Two-pointer tricks

Use **slow/fast** pointers to find the middle (fast moves 2x) or to detect a cycle (Floyd's tortoise and hare — they meet iff a cycle exists). Use a **dummy head** node to simplify edge cases when the head itself might change.

Trees & BSTs

A tree is a hierarchy of nodes with one root and no cycles. A **binary tree** gives each node at most two children. A **binary search tree (BST)** adds an ordering invariant that makes search, insert, and delete O(log n) when balanced.

> **BST invariant**
> For every node, all values in its *left* subtree are smaller, and all values in its *right* subtree are larger. An in-order traversal therefore visits values in sorted order.

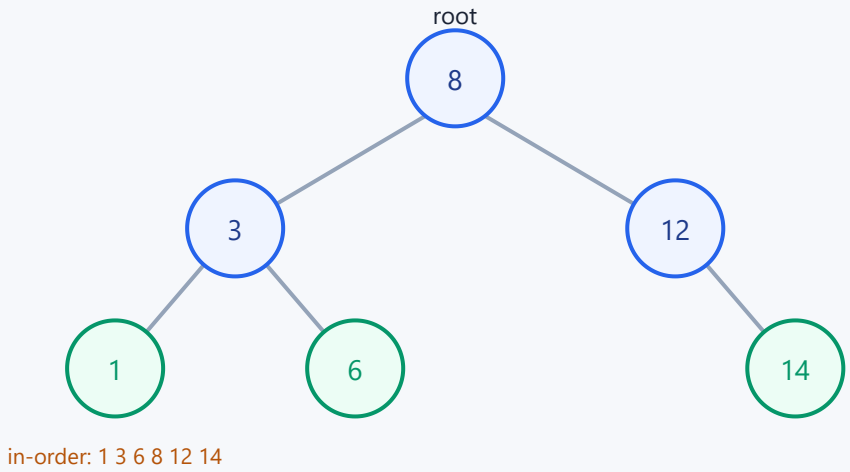


Fig 4. A binary search tree. Left < node < right at every level.

Traversals

Traversal	Order	Typical use
In-order	left, node, right	Sorted output of a BST
Pre-order	node, left, right	Copy/serialize a tree
Post-order	left, right, node	Delete tree, evaluate expressions
Level-order	breadth-first by depth	Shortest path, level grouping

PYTHON

```python
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def inorder(root):
    out = []
    def dfs(node):
        if not node:
            return
        dfs(node.left)
        out.append(node.val)
        dfs(node.right)
    dfs(root)
    return out

def bst_search(root, target):
    while root:
        if target == root.val:
            return root
        root = root.left if target < root.val else root.right
    return None
```

> ⚠ **Watch out**
>
> A plain BST degrades to O(n) if inserts arrive in sorted order (it becomes a linked list). Self-balancing trees — AVL, red-black — keep height O(log n). Python's `dict` / `set` use hashing, but `sortedcontainers.SortedList` gives balanced-tree behavior.

Heaps / Priority Queues

A binary heap is a complete binary tree stored compactly in an array, maintaining the **heap property**: in a min-heap every parent is ≤ its children, so the minimum is always at the root. It gives O(log n) push and pop, and O(1) peek.

WHEN TO REACH FOR A HEAP

"Top k", "k largest/smallest", "merge k sorted lists", running median, Dijkstra's shortest path, scheduling by priority.

ARRAY LAYOUT

For node at index `i`: left child `2i+1`, right child `2i+2`, parent `(i-1)//2`. No pointers needed.

PYTHON

```python
import heapq

# Python's heapq is a MIN-heap
h = []
heapq.heappush(h, 5)
heapq.heappush(h, 1)
heapq.heappush(h, 3)
smallest = heapq.heappop(h)   # 1

# k largest: keep a min-heap of size k
def k_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)   # drop smallest
    return sorted(h, reverse=True)

# Max-heap trick: push negated values
maxh = []
heapq.heappush(maxh, -5)
top = -heapq.heappop(maxh)    # 5
```

> ✦ **Tip**
>
> Building a heap from an existing list with `heapq.heapify(lst)` is O(n), cheaper than n individual pushes (O(n log n)).

Graphs: BFS & DFS

A graph is a set of **vertices** connected by **edges**. Edges may be directed or undirected, weighted or unweighted. The two foundational traversals are breadth-first search (BFS) and depth-first search (DFS).

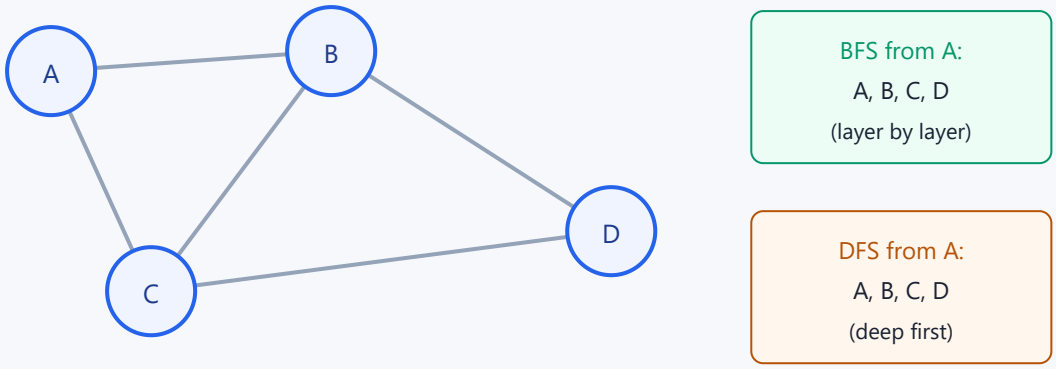


Fig 5. Graph traversal. BFS explores by distance (queue); DFS plunges deep (stack/recursion).

	BFS	DFS
Data structure	Queue	Stack / recursion
Finds	Shortest path (unweighted)	Any path; cycle detection
Memory	O(width)	O(depth)
Time	O(V + E) with an adjacency list	

PYTHON

```python
from collections import deque

def bfs(graph, start):
    visited = {start}
    q = deque([start])
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                q.append(nb)
    return order

def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    for nb in graph[start]:
        if nb not in visited:
            dfs(graph, nb, visited)
    return visited
```

> **Beyond the basics**
> For weighted shortest paths use **Dijkstra** (heap + non-negative weights) or **Bellman-Ford** (handles negatives). For ordering dependencies use **topological sort**. Mark nodes visited *when enqueued* in BFS to avoid duplicates.

Tries

A trie (prefix tree) stores strings by their characters along paths from the root. It gives O(L) insert and lookup where L is the word length — independent of how many words are stored — and makes **prefix** queries natural (autocomplete, spell-check, IP routing).

PYTHON

```python
class Trie:
    def __init__(self):
        self.root = {}
        self.END = '#'

    def insert(self, word):
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node[self.END] = True

    def search(self, word, prefix=False):
        node = self.root
        for c in word:
            if c not in node:
                return False
            node = node[c]
        return True if prefix else self.END in node
```

> **Tip**
> Reach for a trie when a problem involves many strings sharing prefixes, or asks "does any word start with...". A hash set answers full-word membership but can't do prefix queries efficiently.

Core Patterns

Interview problems are variations on a small number of reusable patterns. Recognizing the pattern is 80% of the solve. Here are the six that cover the majority of questions.

Two Pointers

When to use: a sorted array/string, or any time you compare elements from two positions — converging ends, or a slow/fast pair. Turns many $O(n^2)$ scans into $O(n)$.

Show solution — pair summing to target in sorted array

Sliding Window

When to use: contiguous subarray/substring problems — longest/shortest/max-sum window satisfying a constraint. Expand the right edge, shrink the left when the window violates the constraint.

Show solution — longest substring without repeating characters

Binary Search

When to use: a sorted search space, or any monotonic predicate ("smallest x that works"). Halves the space each step for O(log n). Also applies to "binary search on the answer".

Show solution — classic index search

⚠ **Watch out**

The two classic binary-search bugs: an infinite loop (your bounds don't shrink) and off-by-one in the `lo/hi` update. Decide up front whether your interval is inclusive `[lo, hi]` or half-open, and be consistent.

Recursion & Backtracking

When to use: enumerate combinations/permutations/subsets, or explore a decision tree (N-queens, Sudoku, word search). Choose → recurse → *un*choose. Prune branches that can't lead to a solution.

PYTHON

```python
def subsets(nums):
    res, path = [], []
    def backtrack(start):
        res.append(path[:])          # record current subset
        for i in range(start, len(nums)):
            path.append(nums[i])     # choose
            backtrack(i + 1)         # explore
            path.pop()               # un-choose
    backtrack(0)
    return res
```

Dynamic Programming

When to use: optimal substructure + overlapping subproblems. "Count the ways", "min/max cost", "can we reach". Cache subresults (memoization, top-down) or build a table (tabulation, bottom-up).

PYTHON

```python
# Fibonacci: naive O(2^n) → memoized O(n)
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

# Bottom-up, O(1) space
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

> **Tip**
> Define the *state* and the *transition* in words before coding. "dp[i] = max profit using the first i items" is often the whole battle.

Greedy

When to use: a locally optimal choice provably leads to a global optimum (interval scheduling, Huffman coding, some coin systems). Faster than DP when it applies — but you must justify why greedy is safe.

PYTHON

```python
# Activity selection: max non-overlapping intervals
def max_meetings(intervals):
    intervals.sort(key=lambda iv: iv[1])   # by end time
    count, end = 0, float('-inf')
    for s, e in intervals:
        if s ≥ end:
            count += 1
            end = e
    return count
```

Sorting & Searching

You rarely implement a sort in an interview, but you must know their trade-offs and when stability or in-place behavior matters. Python's built-in `sorted()` / `list.sort()` use Timsort: stable, O(n log n), fast on partially sorted data.

Algorithm	Average	Worst	Space	Stable?
Quick sort	O(n log n)	$O(n^2)$	O(log n)	No
Merge sort	O(n log n)	O(n log n)	O(n)	Yes
Heap sort	O(n log n)	O(n log n)	O(1)	No
Timsort (Python)	O(n log n)	O(n log n)	O(n)	Yes
Insertion sort	$O(n^2)$	$O(n^2)$	O(1)	Yes
Counting sort	O(n + k)	O(n + k)	O(k)	Yes

> **Note**
> Comparison-based sorts cannot beat O(n log n) in the worst case. Non-comparison sorts (counting, radix, bucket) can hit O(n) but only for integer-like keys in a bounded range.

Pattern → Signals → Example Problems

Scan the "signals" column when you read a prompt; the words there are your cue to the pattern.

Pattern	Signals in the prompt	Example problems
Two pointers	sorted array, pair, palindrome, in-place	Two Sum II, Valid Palindrome, Container With Most Water
Sliding window	contiguous subarray/substring, longest/shortest, "at most k"	Longest Substring Without Repeating, Min Window Substring
Binary search	sorted, "find smallest/largest that", monotonic	Search in Rotated Array, Koko Eating Bananas
Hash map/set	count, dedupe, "seen before", frequency	Two Sum, Group Anagrams, Subarray Sum Equals K
BFS / DFS	grid, graph, levels, connected, shortest unweighted	Number of Islands, Word Ladder, Course Schedule
Heap	top k, k-th largest, merge k, running median	Kth Largest, Merge k Sorted Lists, Find Median Stream
Backtracking	all combinations/permutations/subsets, "generate all"	Permutations, Combination Sum, N-Queens
Dynamic programming	count ways, min/max cost, "can reach", optimal	Coin Change, Longest Common Subsequence, House Robber

Interview Questions & Answers

Why is a hash map lookup O(1) on average but O(n) in the worst case?

Averaged over a good hash function and load factor, keys spread across buckets so each lookup touches a short chain. In the pathological case where every key hashes to the same bucket, the structure degrades to a single linked list and lookup becomes linear.

When would you choose a linked list over a dynamic array?

When you need frequent insertions/deletions at known positions (especially the head) and don't need random access. Also when you can't tolerate the occasional O(n) resize of a dynamic array, or when implementing structures like LRU caches that splice nodes.

BFS vs DFS — how do you choose?

BFS for shortest path in an unweighted graph and for level-order processing; it uses a queue and O(width) memory. DFS for exhaustive exploration, cycle detection, and topological sorting; it uses recursion/stack and O(depth) memory. On very deep graphs DFS risks stack overflow; on very wide graphs BFS uses more memory.

What makes a problem suitable for dynamic programming?

Two properties: optimal substructure (the optimal answer is built from optimal answers to subproblems) and overlapping subproblems (the same subproblems recur). If subproblems don't overlap, plain divide-and-conquer suffices; if a greedy choice is provably safe, greedy is simpler and faster.

How do you detect a cycle in a linked list in O(1) space?

Floyd's tortoise and hare: advance one pointer by one node and another by two. If they ever meet, there is a cycle; if the fast pointer reaches null, there isn't. It uses constant extra space versus a hash set's O(n).

Why is binary search O(log n), and what must be true to use it?

Each comparison discards half the remaining candidates, so the number of steps is $\log_2(n)$. It requires a sorted structure or, more generally, a monotonic predicate — a point before which the answer is "no" and after which it is "yes".

What is the difference between stable and unstable sorting?

A stable sort preserves the relative order of records with equal keys. This matters when sorting by multiple criteria in passes (e.g., sort by name, then by date keeping name order within a date). Merge sort and Timsort are stable; quicksort and heapsort are not.

How would you find the top k elements in a large stream?

Maintain a min-heap of size k. For each incoming element, push it and, if the heap exceeds k, pop the smallest. This keeps the k largest seen so far in O(n log k) time and O(k) space — far better than sorting everything.

What is amortized analysis, with an example?

It averages the cost of an operation over a sequence rather than looking at the worst single call. A dynamic array's append is O(1) amortized: most appends are O(1), and the occasional O(n) resize is paid for by the many cheap appends before it because capacity doubles.

When is a trie better than a hash set for strings?

When you need prefix queries (autocomplete, "words starting with…") or want to exploit shared prefixes to save memory. A hash set answers exact-membership in O(L) but cannot enumerate by prefix efficiently.

How do you reason about the space complexity of a recursive function?

Count the maximum depth of the call stack (each frame holds locals and the return address) plus any heap allocations that persist. A recursion n levels deep is at least O(n) space even if it allocates nothing else.

Your solution is correct but too slow. How do you improve it?

Identify the bottleneck via complexity analysis, then look for a pattern: replace a nested lookup with a hash map, exploit sortedness with two pointers or binary search, cache repeated subproblems with DP, or prune the search space with better bounds. Always confirm the new complexity before claiming the improvement.

Study Plan & Further Learning

Depth beats breadth. Master the patterns above on a modest set of problems rather than grinding hundreds mechanically.

WEEKS 1–2 — FOUNDATIONS

- Big-O analysis until it is automatic
- Arrays, strings, hash maps, two pointers
- Stacks & queues; sliding window

WEEKS 3–4 — STRUCTURES

- Linked lists (reversal, cycle, merge)
- Trees & BSTs; all four traversals
- Heaps and top-k problems

WEEKS 5–6 — ALGORITHMS

- Graphs: BFS, DFS, topological sort, Dijkstra
- Binary search variants ("on the answer")
- Backtracking templates

WEEKS 7–8 — MASTERY

- Dynamic programming (1D then 2D)
- Greedy and when it is provably safe
- Timed mock interviews, out loud

Prioritized checklist

- Must Explain the complexity of every solution you write.
- Must Solve ~5 problems per pattern until recognition is instant.

- **Must** Practice talking through your approach before coding.
- **Should** Redo problems you failed after one week.
- **Should** Do at least 3 full timed mock interviews.
- **Nice** Study system-design basics if interviewing for senior roles.
- **Nice** Keep a personal "bug log" of mistakes you repeat.

✓ **Final word**

Interviewers hire for clear thinking under pressure, not memorized solutions. A calm, well-communicated process on a familiar pattern beats a silent scramble every time.